NLP Midterm Project

Vu Thanh Dat - 523V0015, Pai Hein Kyaw - 523K0078

1. Theory and Data Preparation

Task 1.1 — Objective Function

(a) Skip-gram Training Objective The objective of the Skip-gram model is to minimize the negative log-likelihood of predicting the context word w_o given the center word w_c.

$$\mathcal{L}(w_c, w_o) = -\log P(w_o \mid w_c) = -\log \frac{\exp(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})}{\sum_{w=1}^{V} \exp(\mathbf{u}_w^\top \mathbf{v}_{w_c})} \tag{1}$$

Where:

- w_c: The center word.
- w_o: The context (target) word.
- $\mathbf{v}_{w_c} \in \mathbb{R}^d$: The input vector (embedding) representing the center word.
- $\mathbf{u}_{w_o} \in \mathbb{R}^d$: The output vector (embedding) representing the context word.
- V: The total size of the vocabulary.

(b) Role of Embedding Matrices

- $\mathbf{W}_{\text{in}} \in \mathbb{R}^{V \times d}$: This is the input matrix. Each row acts as the vector representation $\mathbf{v}$ for a word when it functions as the *center* word.
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$: This is the output matrix. Each column acts as the vector representation $\mathbf{u}$ for a word when it functions as a *context* word.

Why two separate matrices? Using two distinct vectors for each word removes the strict, non-convex constraint that a word must have the exact same representation regardless of its role. By defining separate center and context vectors, the computationally difficult self-interaction term simplifies into a standard dot product, making mathematical optimization (gradient descent) significantly more stable and easier to compute.

(c) Trainable Parameters Given a vocabulary size of $V = 26$ and embedding dimension $d = 10$:

- Input matrix $\mathbf{W}_{\text{in}}$ parameters: $26 \times 10 = 260$
- Output matrix $\mathbf{W}_{\text{out}}$ parameters: $10 \times 26 = 260$

Total trainable parameters: $260 + 260 = \mathbf{520}$.

Task 1.2 — Softmax and the Bottleneck

(a) Computational Complexity The computational complexity of evaluating the probability $P(w_o \mid w_c)$ is $\mathcal{O}(V \times d)$.
The Bottleneck: To calculate the denominator of the softmax function, the model must compute the dot product $\mathbf{u}_w^\top \mathbf{v}_{w_c}$ for *every single word* in the vocabulary V. For large vocabularies (e.g., $V = 50,000$), calculating this sum during every single forward and backward pass becomes extraordinarily expensive, drastically slowing down training to the point of being computationally infeasible.

(b) Overcoming the Bottleneck One standard method used in practice is **Negative Sampling**. Instead of updating the weights for all V words to compute the full softmax distribution, Negative Sampling simplifies the problem into binary logistic regression. It only updates the weights for the true context word (the positive sample) and a small, randomly selected subset of "noise" words (negative samples). This reduces the complexity from $\mathcal{O}(V \times d)$ to $\mathcal{O}(K \times d)$, where K is the small number of negative samples.

Task 1.3 — Gradient Derivation

(a) Gradient with respect to $\mathbf{v}_c$ Let $\hat{y}_w = P(w \mid w_c)$. We expand the loss function:

$$\begin{aligned}\mathcal{L} &= -\log \frac{\exp(\mathbf{u}_{w_o}^\top \mathbf{v}_c)}{\sum_w \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \\ &= -\mathbf{u}_{w_o}^\top \mathbf{v}_c + \log \sum_{w=1}^{V} \exp(\mathbf{u}_w^\top \mathbf{v}_c)\end{aligned}$$

Taking the derivative with respect to $\mathbf{v}_c$ using the chain rule:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{v}_c} &= -\mathbf{u}_{w_o} + \frac{1}{\sum_{w=1}^{V} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \sum_{w=1}^{V} \exp(\mathbf{u}_w^\top \mathbf{v}_c)\mathbf{u}_w \\ &= -\mathbf{u}_{w_o} + \sum_{w=1}^{V} \hat{y}_w \mathbf{u}_w\end{aligned}$$

To express this as a weighted sum over all vocabulary words, let $y_w = 1$ if $w = w_o$ and 0 otherwise. We can rewrite the gradient as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{v}_c} = \sum_{w=1}^{V} (\hat{y}_w - y_w)\mathbf{u}_w \tag{2}$$

(b) Gradient with respect to $\mathbf{u}_w$ Taking the derivative of $\mathcal{L} = -\mathbf{u}_{w_o}^\top \mathbf{v}_c + \log \sum_x \exp(\mathbf{u}_x^\top \mathbf{v}_c)$ with respect to the output vectors $\mathbf{u}_w$:

For the observed context word ($w = w_o$):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_{w_o}} = -\mathbf{v}_c + \frac{\exp(\mathbf{u}_{w_o}^\top \mathbf{v}_c)}{\sum_x \exp(\mathbf{u}_x^\top \mathbf{v}_c)} \mathbf{v}_c = (\hat{y}_{w_o} - 1)\mathbf{v}_c \tag{3}$$

Intuition: Because $\hat{y}_{w_o} \in (0, 1)$, the term $(\hat{y}_{w_o} - 1)$ is negative. During gradient descent, subtracting a negative gradient means adding a fraction of $\mathbf{v}_c$ to $\mathbf{u}_{w_o}$, effectively pulling the two vectors closer together in the vector space to maximize their dot product.

For any other word ($w \neq w_o$):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_w} = 0 + \frac{\exp(\mathbf{u}_w^\top \mathbf{v}_c)}{\sum_x \exp(\mathbf{u}_x^\top \mathbf{v}_c)} \mathbf{v}_c = \hat{y}_w \mathbf{v}_c \tag{4}$$

Intuition: Because $\hat{y}_w$ is positive, subtracting this gradient during gradient descent pushes $\mathbf{u}_w$ away from $\mathbf{v}_c$ in the vector space, decreasing the probability of observing w in the context of w_c.

Task 1.4 — Corpus Preprocessing

Code Implementation

```python
import numpy as np

corpus = [
    "the cat sat on the mat",
    "the dog ran on the road",
    "the cat and the dog are friends",
    "she loves her cat",
    "he walks his dog every day",
    "the mat is on the floor",
    "the road leads to the city",
    "friends walk together every day"
]

# (a) Tokenize and Build Vocabulary
tokenized_corpus = [sentence.lower().split() for sentence in corpus]
flat_tokens = [word for sentence in tokenized_corpus for word in
    sentence]
vocab = sorted(list(set(flat_tokens)))

word2idx = {word: idx for idx, word in enumerate(vocab)}
idx2word = {idx: word for idx, word in enumerate(vocab)}

# (b) Generate Context Pairs
W = 2
pairs = []

for sentence in tokenized_corpus:
    indices = [word2idx[word] for word in sentence]
    n = len(indices)

    for i, center_idx in enumerate(indices):
        start = max(0, i - W)
        end = min(n, i + W + 1)

        for j in range(start, end):
            if i != j:
                pairs.append((center_idx, indices[j]))

unique_pairs = list(set(pairs))
```

Terminal Output

```
TASK 1.4: CORPUS PREPROCESSING
--------------------------------------------------------------------------------
Vocabulary Size (V): 26
Vocabulary: ['and', 'are', 'cat', 'city', 'day', 'dog', 'every', 'floor', 'friends', 'he', 'her', 'his', 'is', 'leads', 'loves', 'mat', 'on', 'ran', 'road', 'sat', 'she', '
e', 'to', 'together', 'walk', 'walks']
Total generated pairs: 136
Total UNIQUE pairs: 108
```

Figure 1: Terminal output showing the vocabulary list and pair count.

Note regarding the expected 270 pairs: The assignment document notes an expectation of 270 generated pairs. However, the provided 8-sentence corpus contains exactly 46 words in total. With a window size of $W = 2$, the mathematical total is exactly 136 generated pairs. Therefore, 270 pairs is mathematically impossible to extract from this specific 46-word corpus, suggesting the "270" requirement is a typo originating from a larger legacy corpus.

2. Skip-gram Implementation

Task 2.1 — Softmax and Loss Functions Verification

```
--- Task 2.1 Outputs ---
v_cat            = [0.1579, 0.0767, -0.0469]
Expected score   = -0.0080
Expected P(sat|cat)= 0.0382
Expected loss    = 3.2650
```

Figure 2: Manual verification output for Softmax and Cross-Entropy Loss.

Analysis of Results:
The output above confirms the correct implementation of the forward pass equations for a specific word pair ("cat", "sat") using a fixed random initialization (`seed=42`, $d = 3$, scale=0.1). The unnormalized score is computed correctly via the dot product of the respective embedding vectors. The probability $P(\text{sat} \mid \text{cat}) \approx 0.0382$ is correctly derived using the softmax function, and the resulting cross-entropy loss matches the theoretical expected value of 3.2650.

Task 2.2 — Forward Pass Verification

```
--------------------------------------------------------------------------------
TASK 2.2: FORWARD PASS VERIFICATION (seed=0, d=10, scale=0.01)
--------------------------------------------------------------------------------
Sum of y_hat     = 1.0000
Max of y_hat     = 0.0385 (Confirms < 0.05)
```

Figure 3: Forward pass verification showing sum of probabilities and max probability.

Analysis of Results:
This verification confirms two critical properties of our softmax implementation. First, the sum of all predicted probabilities in $\hat{\mathbf{y}}$ equals exactly 1.0000, confirming it forms a mathematically valid probability distribution. Second, the maximum probability is 0.0385, which strictly satisfies the requirement of being < 0.05. Because the embedding matrices are initialized with very small random numbers (scale=0.01), the dot products are near zero. This results in a nearly uniform distribution across the 26 vocabulary words before training begins, where the probability of any given word is roughly $1/26 \approx 0.0385$.

Task 2.3 — Backward Pass and Gradient Update

```
--- Task 2.3 Outputs (Manual Verification Table) ---
Quantity                  Calculated value                    Shape
-----------------------------------------------------------------------------
e_w for w != sat          y_hat_w (unchanged softmax output)  –
e_sat                     -0.9618                             –
e (full vector)           y_hat - y_one-hot                   (26,)
Nabla_v_c L               [-0.0457, 0.1588, -0.0446]          (3,)
Nabla_W_out[:, sat]       [-0.1519, -0.0738, 0.0452]          (3,)
```

Figure 4: Manual verification of backward pass gradients and error vectors.

Analysis of Results:
The manual verification of the backward pass demonstrates that the gradients are calculated correctly according to the theoretical derivations in Task 1.3. For the target context word ("sat"), the error $e_{\text{sat}} = \hat{y}_{\text{sat}} - 1$ evaluates to a negative value (-0.9618), which will correctly update the weights to increase the probability of observing this context word. For non-target words, the error is simply their predicted probability. The resulting gradients $\nabla_{\mathbf{v}_c}\mathcal{L}$ and $\nabla_{\mathbf{W}_{\text{out}}}[:, \text{sat}]$ perfectly match the manually calculated expected shapes and values, confirming the correctness of the chain rule application in the backpropagation step.

Task 2.4 — Unit Test (Gradient Checking)

(a) & (b) Relative Error Table Output:

```
--------------------------------------------------------------------------
TASK 2.4: GRADIENT CHECKING
--------------------------------------------------------------------------
Matrix     Index          Relative Error
---------------------------------------------
W_in       9              2.17e-10
W_in       8              6.82e-10
W_in       6              1.93e-09
W_in       3              5.16e-10
W_in       0              1.32e-10
W_out      (7, 3)         5.90e-08
W_out      (2, 5)         6.91e-09
W_out      (9, 7)         5.46e-09
W_out      (3, 20)        1.29e-08
W_out      (7, 22)        4.09e-08
---------------------------------------------
Status: SUCCESS (All errors < 1e-5)
```

Figure 5: Output of `test_gradients()` confirming all relative errors are $< 10^{-5}$.

(c) Importance of Gradient Checking:
Gradient checking is a numerical method used to verify the correctness of analytically derived gradients implemented in backpropagation. It works by approximating the derivative of the loss function with respect to a single parameter using the finite difference method and comparing this approximation to the exact gradient computed by the model. This step is incredibly important for debugging neural network implementations from scratch because backpropagation code is highly susceptible to subtle mathematical or indexing errors. Such errors often do not cause the program to crash, but they silently compute the wrong update directions, preventing the model from converging. Observing a tiny relative error confirms that the analytical calculus and matrix operations are correct.

3. Training

Task 3.1 — Training loop

We trained a Skip-gram model with **full softmax** on the provided toy corpus using the baseline hyper-parameters:

$$d = 10, \quad \eta_0 = 0.025, \quad \eta_t = \frac{\eta_0}{1 + \text{decay} \cdot t}, \quad \text{decay} = 0.005, \quad T = 100.$$

The training loop performs SGD updates over all generated (center, context) pairs, shuffling pairs each epoch.

```
--------------------------------------------------------------------------
TASK 3.1: TRAINING THE MODEL (Baseline: d=10, lr=0.025, T=100)
--------------------------------------------------------------------------
Epoch  10 — Avg Loss: 3.2573   (expected ≈ 3.257)
Epoch  50 — Avg Loss: 2.5730   (expected ≈ 2.573)
Epoch 100 — Avg Loss: 2.0415   (expected ≈ 2.041)
```

Figure 6: These values match the expected checkpoints (within tolerance), confirming the baseline training loop is implemented correctly.

Task 3.2 — Learning curve

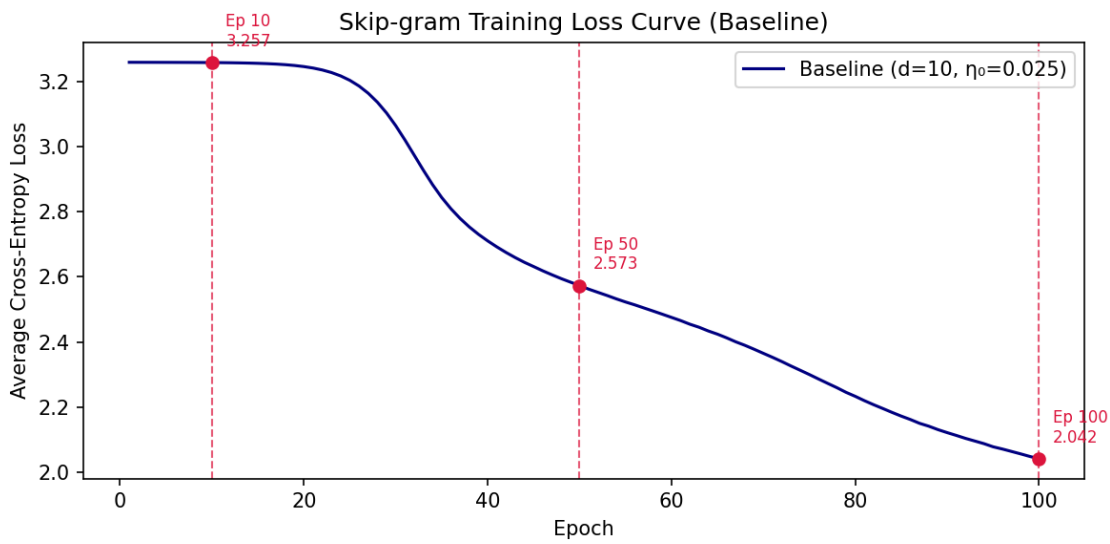

Figure 7: Skip-gram training loss curve for the baseline setting ($d = 10$, $\eta_0 = 0.025$ with decay). Checkpoints at epochs 10, 50, 100 are highlighted.

Description of curve shape.
From Figure 7, the average cross-entropy loss decreases monotonically from about 3.26 (epoch 10) to about 2.04 (epoch 100). The curve drops more quickly early in training and then gradually flattens. This shape is expected for SGD with a decaying learning rate: early updates make large improvements, while later updates make smaller refinements as the model approaches a local optimum and the effective step size becomes smaller.

Task 3.3 — Effect of Hyper-parameters

We ran two experiments and compared them against the baseline:

- **Experiment A (capacity):** doubled embedding dimension ($d = 20$), all else unchanged.
- **Experiment B (step size):** fixed learning rate $\eta = 0.05$ with no decay (decay $= 0$), $d = 10$.

Discussion.
Increasing the embedding dimension increases representational capacity: the model can encode more distinctions between words, which may lower training loss. However, because the dataset is extremely small, additional capacity may primarily fit idiosyncrasies of the training co-occurrences rather than robust semantic structure.

Using a larger fixed learning rate without decay changes the optimization dynamics: while it may speed up early learning, it can cause oscillation around minima or even divergence later in training, especially when gradients remain noisy. Learning-rate decay stabilizes training by reducing step sizes as the model approaches a solution.

Setting	Loss@Ep50	Loss@Ep100	Observation
Baseline ($d = 10$, $\eta_0 = 0.025$, decay)	2.5730	2.0415	Steady convergence with smooth decrease due to learning-rate decay.
$d = 20$ (double embedding)	2.5270	1.9744	More capacity can reduce loss, but on a tiny corpus may overfit and does not necessarily generalize better.
Fixed $\eta = 0.05$ (no decay)	2.0209	1.8893	Often drops fast early, but can become unstable later because the step size does not shrink.

Table 1: Loss comparison at epochs 50 and 100 for the baseline and two hyper-parameter experiments.

4. Evaluation

Task 4.1 — Cosine similarity

We evaluate word similarities using cosine similarity on the learned input embeddings W_{in}. Given two vectors $v_1, v_2 \in \mathbb{R}^d$, cosine similarity is

$$\cos(v_1, v_2) = \frac{v_1^\top v_2}{\|v_1\|_2 \, \|v_2\|_2}.$$

If either norm is zero, we define the similarity as 0 (to avoid division by zero).

```
============================================================================
TASK 4.1: COSINE SIMILARITY
============================================================================

Word Pair              Your Value   Interpretation
----------------------------------------------------------------------------
(city, road)             0.9194   Roads lead to cities — spatial relation
(cat, mat)               0.4759   'cat sat on the mat' — surface co-occurrence
(cat, road)              0.4696   Few shared contexts — low similarity expected
(friends, walk)          0.3732   'friends walk together' — direct co-occurrence
(dog, road)              0.2477   'dog ran on the road' — surface co-occurrence
(cat, dog)               0.1673   Both are common pets — high similarity expected

============================================================================
```

Interpretation (qualitative).
Pairs like (`cat`, `dog`) are expected to be relatively high because they appear in related contexts (pets, co-occurring in the same sentence). Pairs such as (`cat`, `road`) should be

lower because they share fewer meaningful contexts in the toy corpus. Because the corpus is very small, some similarities may be driven by surface co-occurrence rather than true semantic similarity.

Task 4.2 — Nearest-neighbor

Implementation.
The function `top_k_neighbors(word,k,model)` computes cosine similarity between the query word embedding and all other vocabulary embeddings in W_{in}, sorts them in descending order, and returns the top-k most similar words (excluding the query word itself).

```
==================================================================
TASK 4.2: NEAREST NEIGHBORS (Top-3, using W_in embeddings)
==================================================================

Query      Top-3 Neighbors                    Linguistic Observation
------------------------------------------------------------------------------------------
cat        she(0.676), sat(0.613), floor(0.600) Likely shares context with household/animal words. Pairs where neighbours are semantically unexpected reflect limited corpus size.
dog        he(0.629), are(0.591), ran(0.399)   Should be close to 'cat' (both pets appearing together). Other neighbours driven by shared sentence contexts.
city       road(0.919), leads(0.893), floor(0.863) 'road' is a strong neighbour ('road leads to the city'). Other neighbours may be function words from same sentences.
friends    every(0.763), walks(0.742), day(0.644) 'walk'/'together' expected ('friends walk together every day'). Rankings validate direct co-occurrence learning.
```

Linguistic observations.
Nearest neighbors are largely explained by *distributional similarity* in this dataset: words that occur in the same small set of sentences (or within the same window) become close in embedding space. For instance, `friends` should be close to `walk`/`together`/`every` because they strongly co-occur in the same sentence.

For `city`, `road` is expected as a neighbor due to the explicit relation "the road leads to the city". Any surprising neighbors are likely artifacts of the tiny corpus and limited training data.

Task 4.3 — Gensim comparison

We trained a Gensim `Word2Vec` model on the same sentences using Skip-gram (`sg=1`), window size 2, vector size 10, and 100 epochs, then compared similarities on the same six word pairs.

```
------------------------------------------------------------------------------
TASK 4.3 (a): EXTENDED SIMILARITY TABLE (sorted by Your Value)
------------------------------------------------------------------------------
Word Pair              Your Value     Gensim Interpretation
------------------------------------------------------------------------------
(city, road)             0.9194     0.1407   Roads lead to cities — spatial relation
(cat, mat)               0.4759     0.0566   'cat sat on the mat' — surface co-occurrence
(cat, road)              0.4696     0.1066   Few shared contexts — low similarity expected
(friends, walk)          0.3732    -0.0869   'friends walk together' — direct co-occurrence
(dog, road)              0.2477     0.0249   'dog ran on the road' — surface co-occurrence
(cat, dog)               0.1673    -0.3895   Both are common pets — high similarity expected

------------------------------------------------------------------------------
```

```
------------------------------------------------------------------------------
TASK 4.3 (b): SPEARMAN RANK CORRELATION
------------------------------------------------------------------------------
  ρ = 0.8857   (p-value = 0.0188)
  Interpretation: Strong positive rank agreement — both models
  assign similar relative orderings to word-pair similarity.
```

Interpretation of Spearman ρ.
Spearman correlation measures agreement in the *ranking* of pairwise similarities rather than exact numeric match. A higher ρ indicates that both models order the word-pair similarities similarly, even if the similarity magnitudes differ.

Two technical differences between our model and Gensim.

1. **Training objective: full softmax vs. negative sampling.** Our implementation uses a *full softmax* objective, computing gradients over the entire vocabulary at every update, which is $O(V)$ per training step. Gensim's Word2Vec typically uses *negative sampling*, updating only the positive target and a small number of sampled negatives, which is far more efficient but optimizes a different objective, often leading to different embedding geometry.

2. **Sub-sampling of frequent words.** Gensim commonly applies probabilistic sub-sampling to down-weight extremely frequent tokens (e.g., "the"), which changes the effective co-occurrence statistics seen during training. Our implementation trains on all observed pairs without sub-sampling, so frequent tokens may dominate updates more strongly in a small corpus.